

LoRA Fine-Tuning with Unsloth

A practical pipeline for Kaggle GPUs and a local RTX 3060 12GB, from JSONL to a tested, swappable adapter.

Prepare

Choose one base, build clean role data.

Train

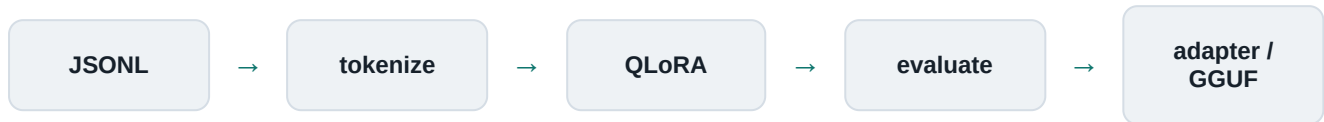
Run the same QLoRA script on Kaggle or local CUDA.

Ship

Evaluate, keep the adapter, export only what serving needs.

From raw examples to a deployable adapter

Start with Qwen3-4B to prove the loop. Move to an 8B model only after the dataset, evaluator, and export path work.



01 LoRA and QLoRA

02 Pick the base model

03 Build the dataset

04 Kaggle setup

05 Training script

06 Kaggle run cycle

07 RTX 3060 path

08 Export and serve

09 Evaluate

10 Troubleshoot

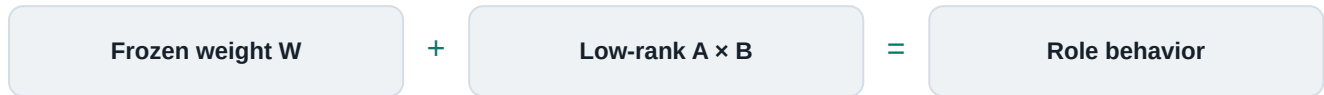
11 Lawyer adapter example

12 Command sheet

Default first run: Qwen3-4B, sequence length 2,048, rank 16, micro-batch 2, accumulation 4, one epoch, 90/10 train/eval split.

LoRA trains the delta, not the whole model

Low-Rank Adaptation (LoRA) freezes the pretrained model and learns small update matrices beside selected linear layers. The original weights stay intact.



LoRA

- The base model normally remains in FP16 or BF16 during training.
- Only adapter parameters receive gradient updates.
- The saved adapter is usually tens to hundreds of megabytes, depending on model size, rank, and target modules.
- At inference, load it beside the matching base or merge it into a standalone model.

QLoRA

- The frozen base is loaded in 4-bit form.
- LoRA parameters still train at a higher working precision.
- Memory drops enough to make 4B to 8B experiments practical on modest GPUs.
- The quality tradeoff is normally smaller than full low-bit training because the base remains frozen.

The mental model: the base carries language and general capability. The adapter nudges behavior, format, vocabulary, and task patterns. It is not a reliable database of current facts.

What the key settings control

Setting	Meaning	Starting point
<code>r</code>	Capacity of each low-rank update	16
<code>lora_alpha</code>	Scale applied to the update	16
<code>target_modules</code>	Linear layers that receive adapters	Attention + MLP projections
<code>load_in_4bit</code>	Quantizes the frozen base	<code>True</code>

Standardize the fleet on one base checkpoint

A LoRA adapter is tied to the architecture and usually the exact base checkpoint it was trained against. A Qwen adapter does not load on Llama, and even a nearby revision can break tensor names or behavior.

Candidate	Best first use	Practical training note
Qwen3-4B Instruct	Fast pipeline proof, tool or role behavior	Easy starting point on 12 to 16 GB
Qwen3-8B	Higher-quality role adapter after proof	Start micro-batch 1 or 2 at 2,048 tokens
Llama 3.1 8B Instruct	Llama ecosystem compatibility	License terms and gated access may apply

Hardware planning

Target	Suggested workload	Starting values
Kaggle T4, 16 GB	4B to 8B QLoRA	batch 2, accumulation 4
Kaggle P100, 16 GB	4B to 8B if the software stack supports Pascal	verify CUDA capability first
Local RTX 3060, 12 GB	4B comfortably, 7B to 8B carefully	batch 1 to 2, checkpointing on

Hardware name: the 12 GB card is the NVIDIA GeForce RTX 3060, not GTX 3060. For an 8B run, batch 2 to 4 is only realistic when sequence length, packing, and data shape cooperate. Start at 1 or 2, then increase after measuring peak VRAM.

Decision rule

1. Choose the smallest instruct model that can pass your baseline tasks.
2. Record the exact model repository and revision in every adapter manifest.
3. Keep tokenizer, chat template, and base revision identical at training and inference.

The dataset decides what the adapter becomes

For a narrow role adapter, 5,000 to 20,000 clean instruction-response pairs is a sensible working range, not a guarantee. Diversity, correctness, and coverage matter more than row count.

Source mix

- **Public-domain corpora:** use only material whose license and provenance you can record.
- **Existing instruction sets:** verify task fit and redistribution terms.
- **Synthetic examples:** generate cheaply with DeepSeek, then filter, deduplicate, and review samples.

Split before tuning

- Reserve 10 percent for evaluation.
- Keep near-duplicates in the same split.
- Create a separate challenge set for edge cases and refusals.
- Never tune against the final certification exam.

Alpaca JSONL

One JSON object per line, UTF-8, with `instruction`, optional `input`, and `output`.

```
{
  "instruction": "Summarize the clause in plain language.",
  "input": "The license terminates upon material breach...",
  "output": "The license can end if a serious breach is not fixed under the agreement's cure terms."
}
{
  "instruction": "Ask the missing questions before answering.",
  "input": "Can I terminate this contract?",
  "output": "Which jurisdiction governs, what termination clause applies, and has either party breached?"
}
{
  "instruction": "Decline a request outside the role.",
  "input": "Guarantee I will win this case.",
  "output": "I cannot guarantee an outcome. I can help identify issues and questions to discuss with a licensed lawyer."
}
```

Quality gates before training

- Parse every line as JSON and require non-empty instruction and output.
- Normalize whitespace, remove exact duplicates, and hash rows for traceability.
- Scan for secrets, personal data, prompt injection, and unlicensed copied text.
- Manually inspect a random sample from every source and category.
- Store `source_id`, license, generator model, and review status outside the training text.

Authenticate once, then run Kaggle as a batch backend

Kaggle CLI authentication is token-based, not OAuth, for this workflow. The legacy `kaggle.json` contains a username and API key, so treat it as a secret.

1

Create the token

In Kaggle, open Settings, find API, choose Create New Token, and download `kaggle.json`. Do not paste its contents into chat, source control, or notebook cells.

2

Install it on the machine that runs the CLI

```
mkdir -p ~/.kaggle
install -m 600 /path/to/downloaded/kaggle.json ~/.kaggle/kaggle.json
python -m pip install --upgrade kaggle
kaggle --version
```

3

Verify without changing anything

```
kaggle kernels list --mine | head
```

Credential rule: add `.kaggle/`, `kaggle.json`, and environment files to ignore rules. Rotate the token if it reaches a repository, notebook output, log, or shared drive.

Budget the run

Free-tier quotas and session ceilings change. A practical planning assumption is roughly 30 GPU-hours per week and a run that must finish within about 9 hours. Check the current Kaggle account quota before launching. Enable Internet only when the notebook must install packages or fetch models.

Package the run as code, metadata, and data

Use a private script kernel for repeatable runs. Keep outputs under `/kaggle/working`, because that directory is collected as kernel output.

Directory layout

```
kaggle-lora/
├── kernel-metadata.json
├── train.py
└── train.jsonl          # or attach a private Kaggle Dataset
```

kernel-metadata.json

```
{
  "id": "YOUR_KAGGLE_USERNAME/qwen3-role-lora",
  "title": "Qwen3 Role LoRA",
  "code_file": "train.py",
  "language": "python",
  "kernel_type": "script",
  "is_private": true,
  "enable_gpu": true,
  "enable_internet": true,
  "dataset_sources": [],
  "competition_sources": [],
  "kernel_sources": []
}
```

Safer data path: upload the JSONL as a private Kaggle Dataset and list it in `dataset_sources`. Then point `DATA_PATH` at its mounted path under `/kaggle/input`. This keeps code and data versioned separately.

First smoke checks inside train.py

```
import os, torch
print("CUDA available:", torch.cuda.is_available())
if torch.cuda.is_available():
    print("GPU:", torch.cuda.get_device_name(0))
    print("Capability:", torch.cuda.get_device_capability(0))
print("Data exists:", os.path.exists(os.environ.get("DATA_PATH", "train.jsonl")))
```

P100 is a Pascal GPU. If the current Kaggle PyTorch, bitsandbytes, or Unsloth build rejects its compute capability, select a T4 or pin a compatible environment rather than repeatedly debugging model code.

Load 4-bit Qwen and format every example with its chat template

In a Kaggle script kernel, install dependencies before imports. On local CUDA, install the matching stack once in the environment instead.

```
# Kaggle only, place before imports
import subprocess, sys
subprocess.check_call([
    sys.executable, "-m", "pip", "install", "-qu",
    "unsloth", "datasets", "trl", "accelerate"
])
```

```
import os, torch
from datasets import load_dataset
from unsloth import FastLanguageModel, is_bfloat16_supported

MODEL = os.getenv(
    "MODEL_ID",
    "unsloth/Qwen3-4B-Instruct-2507-unsloth-bnb-4bit",
)
DATA_PATH = os.getenv("DATA_PATH", "train.jsonl")
OUTPUT = "/kaggle/working/role-lora" if os.path.exists("/kaggle") \
    else "./role-lora"
MAX_SEQ = int(os.getenv("MAX_SEQ_LENGTH", "2048"))

model, tokenizer = FastLanguageModel.from_pretrained(
    model_name=MODEL,
    max_seq_length=MAX_SEQ,
    dtype=None,
    load_in_4bit=True,
)

model = FastLanguageModel.get_peft_model(
    model,
    r=16,
    target_modules=["q_proj", "k_proj", "v_proj", "o_proj",
                    "gate_proj", "up_proj", "down_proj"],
    lora_alpha=16,
    lora_dropout=0,
    bias="none",
    use_gradient_checkpointing="unsloth",
    random_state=3407,
)
```


Format the dataset and define the training run

```

from trl import SFTTrainer, SFTConfig

def to_text(row):
    user = row["instruction"]
    if row.get("input", "").strip():
        user += "\n\nContext:\n" + row["input"].strip()
    messages = [
        {"role": "user", "content": user},
        {"role": "assistant", "content": row["output"]},
    ]
    return {"text": tokenizer.apply_chat_template(
        messages, tokenize=False, add_generation_prompt=False
    )}

data = load_dataset("json", data_files=DATA_PATH, split="train")
data = data.map(to_text)
splits = data.train_test_split(test_size=0.10, seed=3407)

args = SFTConfig(
    output_dir=OUTPUT,
    dataset_text_field="text",
    max_length=MAX_SEQ,
    per_device_train_batch_size=2,
    per_device_eval_batch_size=2,
    gradient_accumulation_steps=4,
    num_train_epochs=1,
    learning_rate=2e-4,
    warmup_ratio=0.03,
    logging_steps=10,
    eval_strategy="steps",
    eval_steps=100,
    save_strategy="steps",
    save_steps=100,
    save_total_limit=2,
    optim="adamw_8bit",
    weight_decay=0.01,
    lr_scheduler_type="linear",
    fp16=not is_bfloat16_supported(),
    bf16=is_bfloat16_supported(),
    packing=True,
    report_to="none",
    seed=3407,
)

```

Train, evaluate, and save the adapter

```
trainer = SFTTrainer(  
    model=model,  
    processing_class=tokenizer,  
    train_dataset=splits["train"],  
    eval_dataset=splits["test"],  
    args=args,  
)  
  
trainer.train()  
metrics = trainer.evaluate()  
print(metrics)  
  
model.save_pretrained(OUTPUT + "/adapter")  
tokenizer.save_pretrained(OUTPUT + "/adapter")
```

Smoke-test before spending hours

1. Temporarily train on 20 to 100 rows.
2. Confirm loss is finite and generally moving down.
3. Confirm evaluation executes and an adapter is written.
4. Reload the saved adapter against the exact base model.
5. Only then launch the complete dataset.

Version drift: TRL occasionally renames configuration arguments. If `max_length` or `eval_strategy` is rejected, inspect `python -c "from trl import SFTConfig; help(SFTConfig)"` in that exact environment, then change only the rejected field.

Effective batch: micro-batch 2 multiplied by accumulation 4 gives an effective batch of 8 examples per optimizer update on one GPU.

Push, inspect, and pull the adapter

Launch the kernel

```
cd kaggle-lora
kaggle kernels push -p .
```

Track the run

```
kaggle kernels status YOUR_KAGGLE_USERNAME/qwen3-role-lora
kaggle kernels logs YOUR_KAGGLE_USERNAME/qwen3-role-lora
```

Download output

```
mkdir -p ./kaggle-output
kaggle kernels output \
  YOUR_KAGGLE_USERNAME/qwen3-role-lora \
  -p ./kaggle-output
```

Confirm the adapter is complete

```
find ./kaggle-output -maxdepth 3 -type f -printf '%P\n' | sort
# Expect adapter_config.json, adapter_model.safetensors,
# tokenizer files, and trainer/checkpoint metadata.
```

Before a full run

- Run 20 to 100 examples.
- Set one epoch.
- Confirm GPU name and peak memory.
- Confirm output files download.

Before quota resets

- Pull every completed output.
- Keep mid-run checkpoints.
- Record the kernel version.
- Do not rely on session storage.

Time estimate: “A few hours” for 10,000 examples on an 8B model is only a planning estimate. Token count, packing, GPU assignment, evaluation cadence, and sequence length dominate runtime. Measure a 100-step sample and extrapolate.

Use the NucBox for fast iteration and private data

The RTX 3060 12GB is a good local QLoRA target. Prove the environment with Qwen3-4B, then test an 8B model with conservative memory settings.

Create an isolated environment

```
nvidia-smi
python3 -m venv .venv
source .venv/bin/activate
python -m pip install -U pip
python -m pip install -U torch
python -m pip install -U unsloth datasets trl accelerate
python - <<'PY'
import torch
print("torch", torch.__version__)
print("CUDA build", torch.version.cuda)
print("available", torch.cuda.is_available())
print("GPU", torch.cuda.get_device_name(0) if torch.cuda.is_available() else None)
PY
```

Use the PyTorch installer selector if the generic wheel does not match the installed NVIDIA driver. A locally installed CUDA toolkit does not have to equal `torch.version.cuda`, but the driver must support the wheel.

Run the same script

```
export DATA_PATH="$PWD/train.jsonl"
export MODEL_ID="unsloth/Qwen3-4B-Instruct-2507-unsloth-bnb-4bit"
export MAX_SEQ_LENGTH=2048
python train.py
```

Watch memory and thermals

```
watch -n 1 nvidia-smi
```

Keep the two backends identical: same data hash, model revision, seed, tokenizer, chat template, rank, learning rate, and split. Change only the settings required by hardware.

Reduce sequence memory before reducing adapter capacity

When an 8B run fails on 12 GB, make one change at a time in this order.

1. **Drop micro-batch to 1.** Preserve the effective batch with more gradient accumulation.
2. **Shorten sequence length.** Move from 2,048 to 1,536 or 1,024. Attention memory grows rapidly with sequence length.
3. **Keep gradient checkpointing on.** "unsloth" trades extra compute for lower activation memory.
4. **Use the 8-bit optimizer.** Keep `optim="adamw_8bit"`.
5. **Disable evaluation during the smoke run.** Restore it for the real run.
6. **Only then lower rank.** Try `r=8` if capacity is still too expensive.

Known-good starting grid

Model	Sequence	Micro-batch	Accumulation
Qwen3-4B	2,048	2 to 4	2 to 4
Qwen3-8B	2,048	1 to 2	4 to 8
8B, OOM fallback	1,024	1	8 to 16

Do not promise runtime from row count alone: 10,000 examples could mean 2 million or 20 million tokens. Run 100 steps, record seconds per step, include evaluation and saving overhead, then estimate. "A few hours" is plausible for a tuned 8B run, not guaranteed.

Reproducibility record

```
nvidia-smi --query-gpu=name,driver_version,memory.total --format=csv
python -m pip freeze > environment.txt
sha256sum train.jsonl > data.sha256
```

Keep the adapter as the source artifact

The PEFT adapter is the smallest, most flexible deliverable. Preserve it even if you also produce a merged model or GGUF.

Option A, publish the adapter to Hugging Face Hub

```
# Authenticate without putting the token in source
hf auth login

# In Python, after training
model.push_to_hub("YOUR_USER/qwen3-role-lora")
tokenizer.push_to_hub("YOUR_USER/qwen3-role-lora")
```

Option B, export a standalone GGUF for LM Studio

```
# In Python, merges the active adapter for a standalone file
model.save_pretrained_gguf(
    "./role-model-gguf",
    tokenizer,
    quantization_method="q4_k_m",
)
```

Import the resulting GGUF into LM Studio. Use the same chat template used during training; a mismatch can make a good adapter look broken.

Option C, hot-swap the adapter

```
# Convert a PEFT adapter with the current llama.cpp converter
python /path/to/llama.cpp/convert_lora_to_gguf.py \
    ./role-lora/adapter --outfile ./role-lora.gguf

# Run beside the matching base GGUF
llama-cli -m ./matching-base.gguf --lora ./role-lora.gguf
```

```
# vLLM serves PEFT adapters beside the matching Hugging Face base
python -m vllm.entrypoints.openai.api_server \
    --model YOUR_EXACT_BASE_ID \
    --enable-lora \
    --lora-modules role=./role-lora/adapter
```

Choose deliberately: adapter for reuse and hot-swap, merged GGUF for a single-file local app, Hub repository for sharing and versioning. Record the base ID in all three.

A lower training loss is not a release decision

Compare the adapter with the untouched base on examples neither model saw during training.

Minimum release gate

Check	Measure	Pass condition
Held-out loss	Evaluation split	Plateaus or improves without divergence
Task score	Exact or rubric score	Beats the base on target tasks
Regression set	General instructions	No material capability collapse
Open-ended quality	Blind human rubric	Stable preference over base
Safety / boundaries	Adversarial prompts	Required refusals and escalation hold

Human-judge agreement

For open-ended work, have two reviewers score a sample using the same rubric. Track agreement, adjudicate disagreements, and update the rubric before changing the model. An LLM judge can accelerate triage, but calibrate it against human labels.

When to stop

- Stop when evaluation loss plateaus while training loss keeps falling.
- Stop when target-task gains flatten across checkpoints.
- Stop immediately if general-task regressions or unsafe confident answers increase.
- Keep the best checkpoint by evaluation result, not automatically the last checkpoint.

Certification rule: training data, development prompts, and the final Agent School exam must be separate. Otherwise the certificate measures memorization.

Fix the environment or memory bottleneck before tuning hyperparameters

Symptom	Likely cause	First response
CUDA out of memory	Sequence or micro-batch too large	Batch 1, then shorter sequence
<code>torch.cuda.is_available()</code> false	Driver or wrong PyTorch wheel	Check <code>nvidia-smi</code> and reinstall matching wheel
Unsupported GPU capability	P100 with incompatible current build	Use T4 or compatible pinned stack
Kaggle run stops	Session ceiling or quota	Shorten run, save checkpoints, resume
Model download is slow	Cold cache or Internet setting	Enable Internet, attach cached model data
Adapter will not load	Wrong architecture or base revision	Match exact base ID and tokenizer
Output loops or degrades	Template mismatch or overfit	Match chat template, inspect earlier checkpoint
Loss is <code>nan</code>	Precision, bad rows, or high LR	Validate data, lower LR, check FP16/BF16

Fast diagnostics

```
# GPU and process state
nvidia-smi
python -c "import torch; print(torch.__version__, torch.version.cuda, torch.cuda.is_available())"

# Dataset integrity
python - <<'PY'
import json
for n, line in enumerate(open('train.jsonl', encoding='utf-8'), 1):
    x = json.loads(line)
    assert x.get('instruction') and x.get('output'), n
print('JSONL OK')
PY

# Adapter identity
python - <<'PY'
import json
x=json.load(open('role-lora/adapter/adapter_config.json'))
print(x.get('base_model_name_or_path'), x.get('r'), x.get('target_modules'))
PY
```

Debugging discipline: preserve the failing command, package versions, GPU name, and first complete traceback. Do not “fix” several settings at once.

A lawyer-role adapter needs retrieval, boundaries, and human review

Use the adapter to shape legal issue-spotting, plain-language explanation, drafting structure, and uncertainty behavior. Do not use it as the source of current law.

Training mix

60 PERCENT

Legal instruction pairs

Issue spotting, clause explanation, question generation, memo structure, and jurisdiction-aware analysis.

20 PERCENT

Boundaries and refusal

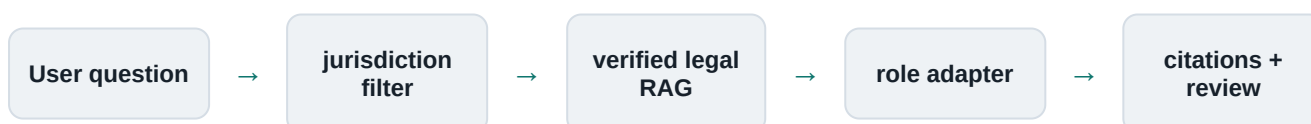
"I am not a licensed attorney," missing-fact questions, no outcome guarantees, emergency escalation.

20 PERCENT

General instruction

Retain ordinary following, summarization, formatting, and tool-use behavior.

Inference architecture



- **Retrieval supplies facts:** statutes, regulations, cases, effective dates, and source URLs.
- **The adapter supplies behavior:** analysis pattern, tone, structured output, and boundary responses.
- **The application verifies citations:** reject any citation not present in retrieved sources.
- **A licensed professional reviews consequential output:** especially filings, deadlines, strategy, and jurisdiction-specific advice.

Known failure mode: a fluent legal model can invent plausible case names and holdings. LoRA cannot solve that. Require source-grounded generation and a fail-closed citation checker.

The complete command path

Authenticate Kaggle

```
mkdir -p ~/.kaggle
install -m 600 kaggle.json \
  ~/.kaggle/kaggle.json
python -m pip install -U kaggle
kaggle kernels list --mine | head
```

Local environment

```
nvidia-smi
python3 -m venv .venv
source .venv/bin/activate
python -m pip install -U pip torch
python -m pip install -U \
  unsloth datasets trl accelerate
```

Push and inspect

```
kaggle kernels push -p kaggle-lora
kaggle kernels status \
  USER/qwen3-role-lora
kaggle kernels logs \
  USER/qwen3-role-lora
```

Run locally

```
export DATA_PATH="$PWD/train.jsonl"
export MODEL_ID="unsloth/Qwen3-4B-Instruct-
2507-unsloth-bnb-4bit"
export MAX_SEQ_LENGTH=2048
python train.py
```

Download output

```
kaggle kernels output \
  USER/qwen3-role-lora \
  -p ./kaggle-output
```

Inspect and preserve

```
find role-lora/adapters -type f -maxdepth 2
sha256sum train.jsonl > data.sha256
python -m pip freeze > environment.txt
```

Validate JSONL

```
python - <<'PY'
import json
for n,l in
enumerate(open('train.jsonl'),1):
  x=json.loads(l)
  assert x.get('instruction') and
  x.get('output'),n
  print('OK')
PY
```

Serve with llama.cpp

```
llama-cli \
  -m matching-base.gguf \
  --lora role-lora.gguf
```

First-run defaults: Qwen3-4B, 2,048 tokens, rank 16, alpha 16, batch 2, accumulation 4, one epoch, 90/10 split. If OOM: batch 1, then 1,024 tokens. If quality fails: inspect data before increasing rank or epochs.

Sources and further reading

Product APIs and quotas change. Recheck the current documentation before pinning an environment or budgeting a long run.

1. **Hugging Face PEFT, LoRA conceptual guide.** Parameters, target modules, scaling, and adapter behavior.
https://huggingface.co/docs/peft/main/conceptual_guides/lora
2. **Hugging Face course, LoRA configuration.** Practical rank, alpha, dropout, and target-module discussion.
<https://huggingface.co/docs/course/chapter11/4>
3. **Unsloth Qwen3 model collection.** Published Qwen3 variants and quantized checkpoints.
<https://huggingface.co/collections/unsloth/qwen3>
4. **Unsloth Qwen3-4B Instruct 2507, 4-bit checkpoint.** Exact model used in the guide's default command.
<https://huggingface.co/unsloth/Qwen3-4B-Instruct-2507-unsloth-bnb-4bit>
5. **Qwen3-4B model card.** Model size, architecture details, and context information.
<https://huggingface.co/Qwen/Qwen3-4B>
6. **vLLM, using LoRA adapters.** Serving adapters with `--enable-lora` and `--lora-modules`.
<https://docs.vllm.ai/en/v0.5.2/models/lora.html>
7. **llama.cpp, GGUF LoRA example.** Loading a base model and adapter with `--lora`.
<https://github.com/ggml-org/llama.cpp/discussions/10123>
8. **llama.cpp, PEFT-to-GGUF conversion change.** Current converter behavior and an example command.
<https://github.com/ggml-org/llama.cpp/pull/10110>

REFERENCES

Kaggle operations and practical hardware notes

9. **Kaggle CLI workflow reference.** Kernel initialization, push, status, and output retrieval.
<https://github.com/mederic-hurier-partner/dotfiles/blob/HEAD/skills/kaggle/SKILL.md>
10. **Kaggle notebook CLI example.** Legacy `kaggle.json` location, permissions, kernel push, and GPU metadata.
<https://github.com/hikarusedashi/kaggle-notebooks>
11. **Kaggle integration field notes.** Approximate free-tier GPU quota and session assumptions, with the warning that limits can change.
https://github.com/tayloramareltech/gemma4_comp/blob/HEAD/docs/kaggle_integration.md
12. **RTX 3060 Qwen3-8B field guide.** A practical 12 GB Unsloth configuration and memory estimates, used only as a starting point to verify locally.
https://github.com/millerrc18/halcyon-lab/blob/HEAD/docs/research/Fine-Tuning_Qwen3_8B_RTX_3060_March_2026_Guide.md

Source hierarchy: model cards and library documentation establish interfaces. Community field notes establish practical starting points, not guarantees. Verify the current package help and Kaggle account limits before each run.

Adapter manifest fields to keep

```
{
  "base_model": "exact repository and revision",
  "dataset_sha256": "...",
  "chat_template": "tokenizer-defined",
  "rank": 16,
  "alpha": 16,
  "max_seq_length": 2048,
  "seed": 3407,
  "training_backend": "kaggle-t4 or local-rtx3060",
  "evaluation_report": "path or URL"
}
```